ADVANCED DATA ANALYSIS · MODULE III

Machine Learning I

Essentials, evaluation, and tuning

M. Fuat Kına

ROADMAP

What this session covers

01	**What machine learning is**	Prediction versus explanation; where it departs from statistics
02	**Supervised learning**	Classification with k-NN; regression and the loss function
03	**What is genuinely new**	Cross-validation, regularisation, the bias–variance trade-off
04	**Evaluation and tuning**	Class imbalance, precision–recall, ROC/AUC, grid search
05	**Preprocessing and pipelines**	Imputation, encoding, scaling — assembled into one object
06	**Unsupervised learning**	k-means clustering, inertia, and the elbow

Lab in Python throughout: scikit-learn, on a dataset we clean together.

DEFINITION

What is machine learning?

Teaching computers to perform a task by learning from data, rather than by being explicitly programmed to do it.

- Giving a machine the ability to learn to make decisions.
  - Is this email spam, given its content and its sender?
  - Which Wikipedia entries group together, given the words they contain?
- From statistics to machine learning: from fitting a model to a sample, to predicting the world — nowcasting and forecasting.
- Labels present → supervised learning. No labels → unsupervised learning.

A SHORT HISTORY

1980s

The models are worked out — and mostly unusable.

2000s

Compute and data arrive; machine learning becomes practice.

2010s

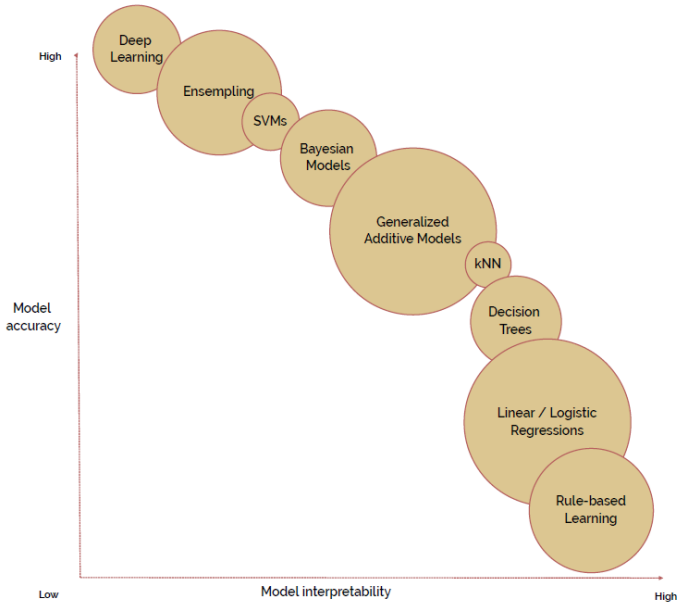
Deep learning and neural networks.

2020s

Generative models.

DEFINITION

What is machine learning?



A SHORT HISTORY

1980s

The models are worked out — and mostly unusable.

2000s

Compute and data arrive; machine learning becomes practice.

2010s

Deep learning and neural networks.

2020s

Generative models.

As models grew more complex, predictive capacity rose and interpretability fell. Figure reference: Toosi et al. (2022).

FRAMING

Prediction is not explanation

Explanation

Which variables cause the outcome, and by how much? The coefficient is the object of interest. In-sample fit, standard errors, identification.

Ask: is β unbiased?

Prediction

How accurately can I label an unseen case? The prediction is the object of interest; the coefficients may be uninterpretable. Out-of-sample error.

Ask: does it generalise?

A model can predict well and explain nothing — and, less often noticed, explain well and predict badly. Module III/Week 10 returns to this with the causal machine learning literature.

Reading: Molina & Garip (2019); Hofman et al. (2021).

1

Supervised Learning

Labelled data, a target to predict, and an error you can measure

TWO FAMILIES

Supervised and unsupervised learning

UNSUPERVISED · UNLABELLED

- Uncovering hidden patterns and structure in data that carries no labels.
- Clustering, dimension reduction.
- There is no right answer to score against — evaluation is indirect.

SUPERVISED · LABELLED

- Learning from data whose correct output is known, so as to predict the output for new data.
- Automates an expensive manual task, or forecasts.
- Requires labels: historical records, crowdsourced coding, expert annotation.

The distinction is about the data you hold, not about the algorithm you like.

VOCABULARY

The supervised setup, in words you will need

- Use predictor variables (features) and a target variable to build a model that predicts the target.
- Feature = independent variable = predictor. Target = dependent variable = response = outcome.
- Continuous target → regression. Binary or categorical target → classification.

```
from sklearn.neighbors import KNeighborsClassifier

X = df[features]      # n × p matrix
y = df['target']      # n × 1 vector

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
y_hat = model.predict(X_test)
```

TWO STEPS, ALWAYS

`.fit()`

Learning. The model sees features and labels and adjusts its parameters.

`.predict()`

Predicting. The fitted model assigns labels to data it has never seen.

Everything else

Preprocessing, validation, tuning — the work that makes those two lines trustworthy.

The scikit-learn API is deliberately uniform: every estimator answers to the same two verbs.

QUICK CHECK

Which of these is a classification problem?

A Using labelled financial data to predict whether a stock will go up or down next week.

B Using labelled housing data to predict the price of a new house from its features.

C Using unlabelled data to cluster the students of an online school by learning style.

D Using labelled financial data to predict what the value of a stock will be next week.

Two of these are supervised. Only one has a discrete target.

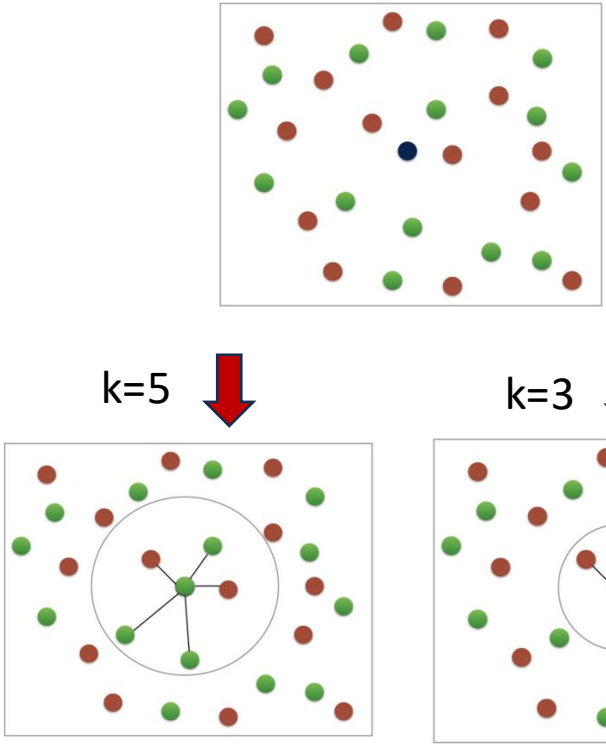
1.A Classification

k-nearest neighbours, decision boundaries, and model complexity

k-nearest neighbours

- Look at the k closest labelled points and take a majority vote.
- This draws a decision boundary between the classes — implicitly, without any equation for it.
- The boundary is where the vote flips. Change k and the boundary moves.
- No training in the usual sense: the model is the data. Prediction is a distance computation.

k = 5 and k = 3 can label the same point differently. That is not a bug; it is the hyperparameter doing its job.



Accuracy, and where to measure it

- Accuracy = the fraction of predictions that are correct.
- On which data should I compute it — the data I fitted, or data the model has not seen?
- Fitted-data accuracy answers the wrong question: it rewards memorising.
- So: split. Fit the classifier on the training set, predict on the test set, score there.

```
from sklearn.model_selection import train_test_split

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42,
    stratify=y)

knn.fit(X_tr, y_tr)
knn.score(X_te, y_te)
```

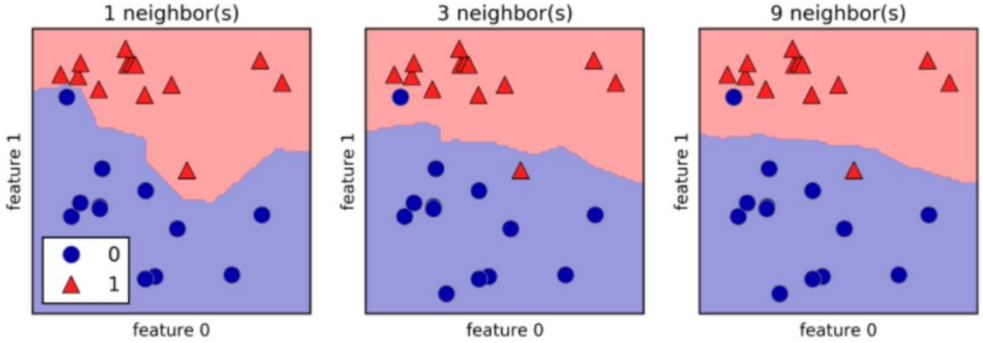
ONE SPLIT



Stratify when classes are unbalanced
so the test set keeps the same class proportions as the population. Fix random_state so your split — and therefore your reported number — is reproducible by someone else.

The test set is spent the moment you look at it. Tune on validation data, not on the test set.

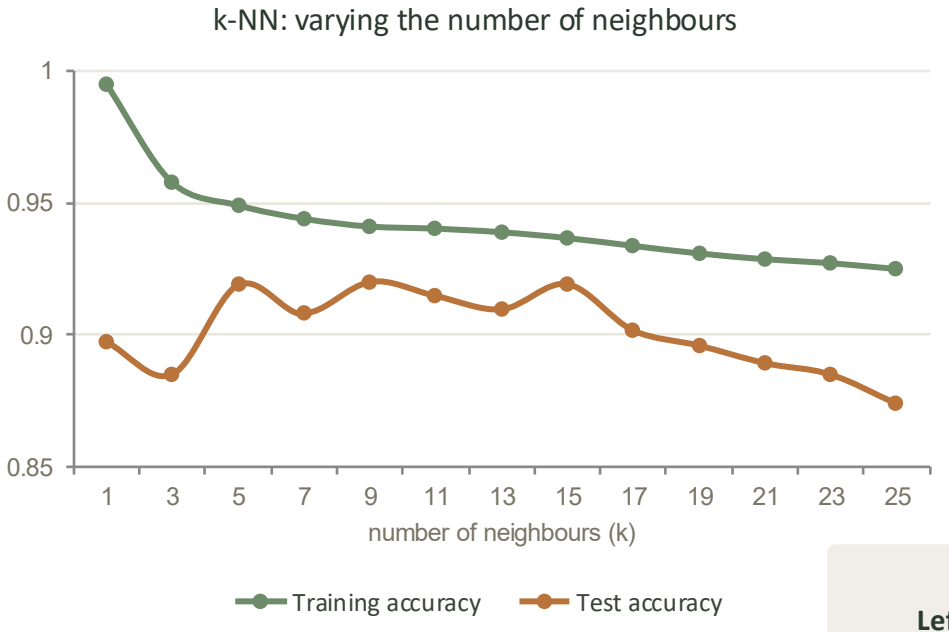
Larger k, smoother boundary, simpler model



- Larger k → smoother decision boundaries → less complex model.
- Smaller k → more complex model → can overfit.
- Training accuracy falls monotonically in k; test accuracy peaks somewhere in the middle. That peak is what you are looking for.

MODEL COMPLEXITY

Larger k, smoother boundary, simpler model



Left of the peak: overfitting.
Right of the peak: underfitting.

1.B Regression

A continuous target, a loss function, and the ideas that make it machine learning

QUICK CHECK

Which of these is a regression problem?

1. An e-commerce firm using labelled customer data to predict whether a customer will buy a given item.

2. A health company using tumour measurements to predict whether a new tumour is benign or malignant.

3. A restaurant using review text to assign positive or negative sentiment to a review.

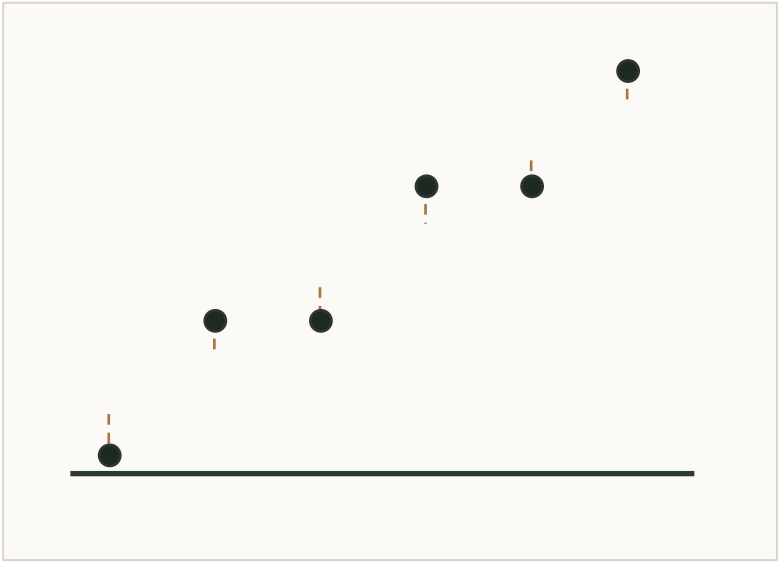
4. A bike-share company using time and weather data to predict how many bikes are rented in a given hour.

The question is never the domain. It is the type of the target variable.

The line that minimises the loss

$$y = a\,x + b$$

- We want the line that makes the error — the loss — as small as possible.
- OLS minimises the sum of squared residuals. That is a choice of loss, not a law of nature.
- Everything that follows keeps this skeleton and changes the loss.



RESIDUALS — SQUARED, THEN SUMMED

Lab: run the classical OLS model with scikit-learn, then compare it with statsmodels output.

THE REAL CONTENT

What is actually new in a machine learning model?

Linear regression is two centuries old. What machine learning adds is a discipline around it.

01

Cross-validation

Estimating out-of-sample error without spending your test set.

02

Regularisation

Ridge and lasso: paying for complexity, in the loss function itself.

03

Fine-tuning

Metrics beyond accuracy; the ROC curve and the area under it.

04

Hyperparameters

Values the data cannot estimate for you — searched, not fitted.

05

Preprocessing

Imputation, encoding, scaling — as part of the model, not before it.

06

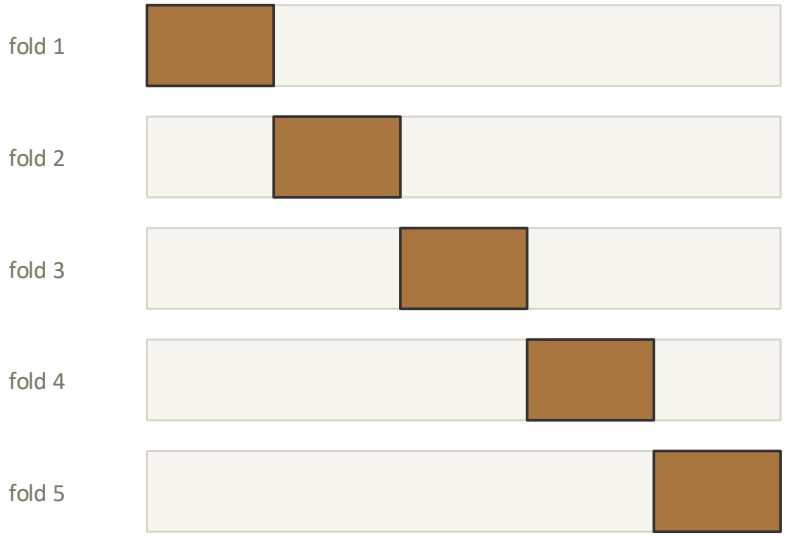
Pipelines

The whole sequence as one object you can cross-validate and ship.

Cross-validation

- A single train/test split gives one number — and that number depends on which rows happened to fall where.
- k-fold: split the training data into k folds, hold each out in turn, average the k scores.
- You get a mean and a spread. Report both; a model that is good on average and unstable is not good.
- Cost: k fits instead of one. This is why cross-validation and grid search together get expensive fast.

```
from sklearn.model_selection import cross_val_score
cv = cross_val_score(model, X, y,
                     cv=5, scoring='f1_macro')
cv.mean(), cv.std()
```



GOLD = HELD OUT FOR VALIDATION

REGULARISATION

Ridge and lasso: penalising the coefficients

Unpenalised OLS is a perfect overfitting machine when p is large. Regularisation adds a price for complexity to the loss.

RIDGE · L2

$$\alpha \cdot \Sigma\, a_i^2$$

- Loss = OLS loss + ridge penalty
- α controls model complexity
- α = 0 gives you OLS back (overfitting)
- Very large α → underfitting
- Shrinks coefficients towards zero, keeps them all

LASSO · L1

$$\alpha \cdot \Sigma\, |a_i|$$

- Loss = OLS loss + lasso penalty
- α controls model complexity
- α = 0 gives you OLS back (overfitting)
- Very large α → underfitting
- Sets some coefficients exactly to zero — selection

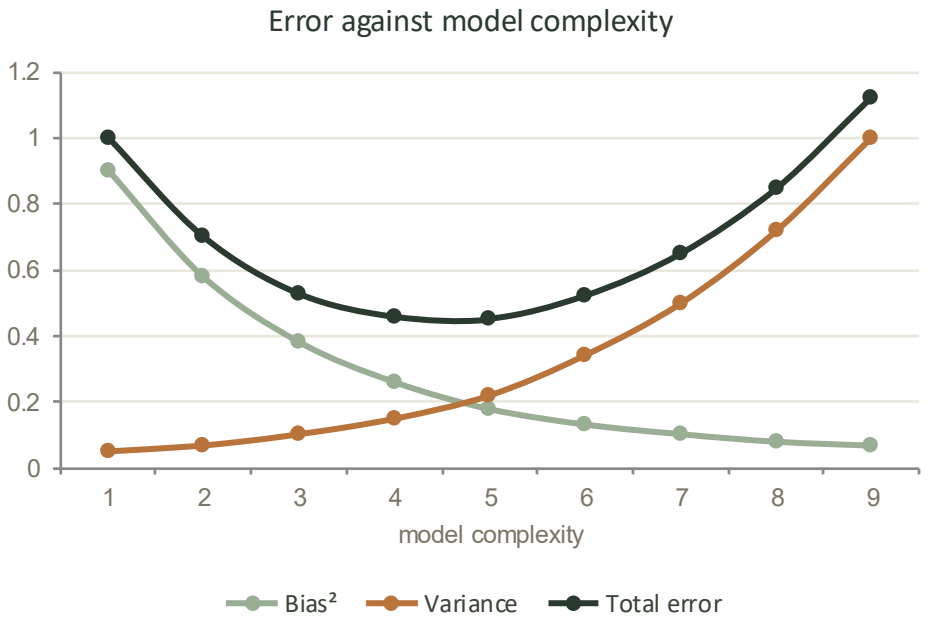
Lasso is also a blunt but useful reader of feature importance: what survives the penalty carried weight.

THE TRADE-OFF

Bias and variance

- Bias: the model is too rigid to represent the pattern — it misses systematically. Underfitting.
- Variance: the model chases the noise in this particular sample — it will not repeat. Overfitting.
- Total error falls, then rises. The minimum is the model complexity you want, and regularisation is how you steer towards it.

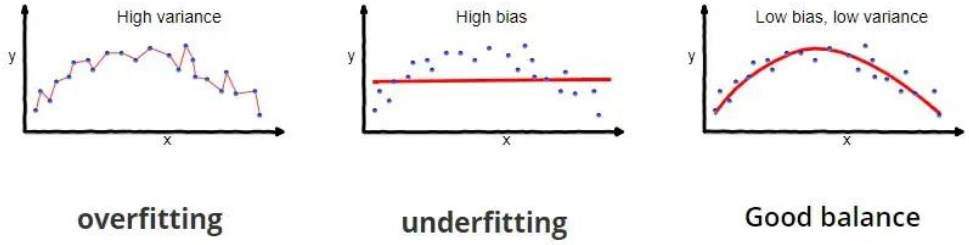
α is the dial. Cross-validation is how you read the dial.



THE TRADE-OFF

Bias and variance

- Bias: the model is too rigid to represent the pattern — it misses systematically. Underfitting.
- Variance: the model chases the noise in this particular sample — it will not repeat. Overfitting.
- Total error falls, then rises. The minimum is the model complexity you want, and regularisation is how you steer towards it.



α is the dial. Cross-validation is how you read the dial.

1.C

Performance and Tuning

Class imbalance, the confusion matrix, ROC/AUC, and hyperparameter search

EVALUATION

Why accuracy lies

- Spam classification: 1% of the mail is spam, 99% is real.
- Label everything 'real' and the model is 99% accurate — and completely useless.
- This is class imbalance, and it is the normal case in social science data.
- So we need metrics that distinguish the kinds of mistake.

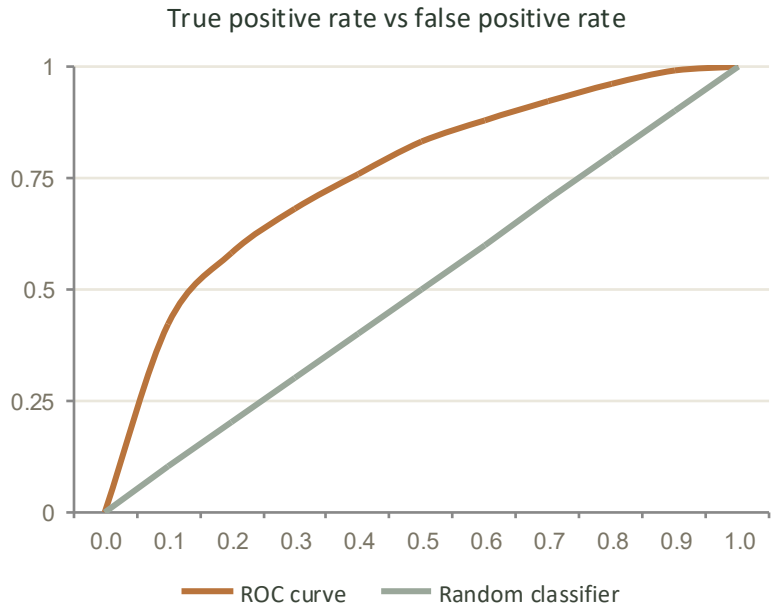
	Predicted spam	Predicted real
Actual: spam	True positive	False negative
Actual: real	False positive	True negative

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
F1 = 2 · (precision · recall) / (precision + recall)
```

Which metric matters is a substantive question: what does a false positive cost you, and who pays it?

Logistic regression, the ROC curve, and AUC

- Logistic regression classifies by predicting a probability: label 1 if p > 0.5.
- But 0.5 is a convention, not a result. Vary the threshold and precision and recall move against each other.
- The ROC curve traces true positive rate against false positive rate as the threshold sweeps from 1 to 0.
- AUC — the area under that curve — summarises the model across all thresholds. 0.5 is a coin flip; 1.0 is perfect.

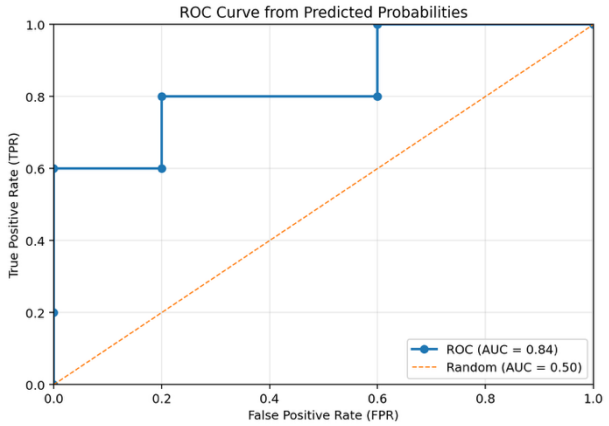



```
from sklearn.metrics import roc_curve,
roc_auc_score
p = model.predict_proba(X_te)[:, 1]
fpr, tpr, thr = roc_curve(y_te, p)
roc_auc_score(y_te, p)
```

Logistic regression, the ROC curve, and AUC

A = 0 (negative)
B = 1 (positive)
Real positive count P = 5
Real negative count N = 5

Index	1	2	3	4	5	6	7	8	9	10
Real class	A	A	B	A	A	B	A	B	B	B
Predicted prob.	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1



Threshold (t)	Positive predicted (score ≥ t)	TP	FP	TPR	FPR
>1.0	–	0	0	0.00	0.00
≥1.0	10	1	0	0.20	0.00
≥0.9	9–10	2	0	0.40	0.00
≥0.8	8–10	3	0	0.60	0.00
≥0.7	7–10	3	1	0.60	0.20
≥0.6	6–10	4	1	0.80	0.20
≥0.5	5–10	4	2	0.80	0.40
≥0.4	4–10	4	3	0.80	0.60
≥0.3	3–10	5	3	1.00	0.60
≥0.2	2–10	5	4	1.00	0.80
≥0.1	1–10	5	5	1.00	1.00

TUNING

Hyperparameters are searched, not estimated

- Ridge and lasso: which α? k-NN: which n_neighbors? Forests: how deep?
- These are hyperparameters — the data cannot hand them to you the way it hands you a coefficient.
- Procedure: try a set of values, fit each separately with cross-validation, compare, keep the best.

Grid search CV

Every combination on a grid you specify. Exhaustive, reproducible, and exponential in the number of hyperparameters.

Randomized search CV

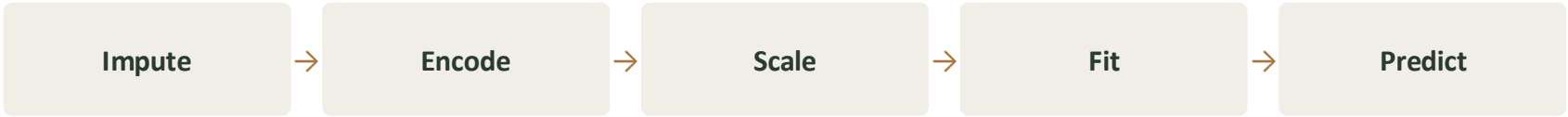
Samples combinations at random from distributions. Usually finds a comparable optimum for a fraction of the compute.

```
grid = {'alpha': np.logspace(-4, 2, 20)}
search = GridSearchCV(Ridge(), grid, cv=5, scoring='neg_mean_squared_error')
search.fit(X_tr, y_tr);  search.best_params_, search.best_score_
```

Real data does not arrive ready for the model

- Categorical features → dummies. OneHotEncoder in scikit-learn; get_dummies() or pd.cut() in pandas.
- Missing data: dropping rows loses information and, usually, loses it non-randomly. Impute instead.
  - Mean or median of the observed entries; linear interpolation if there is a time dimension.
- Scaling: distance-based methods (k-NN, SVM) and penalised regressions are not scale-invariant.

A pipeline is not tidiness. It is what stops the test set leaking into your imputer.




```
pipe = Pipeline([('imp', SimpleImputer(strategy='median')),
                 ('sc',  StandardScaler()),
                 ('clf', LogisticRegression(max_iter=1000))])
cross_val_score(pipe, X, y, cv=5)   # every step refitted inside every fold
```

2

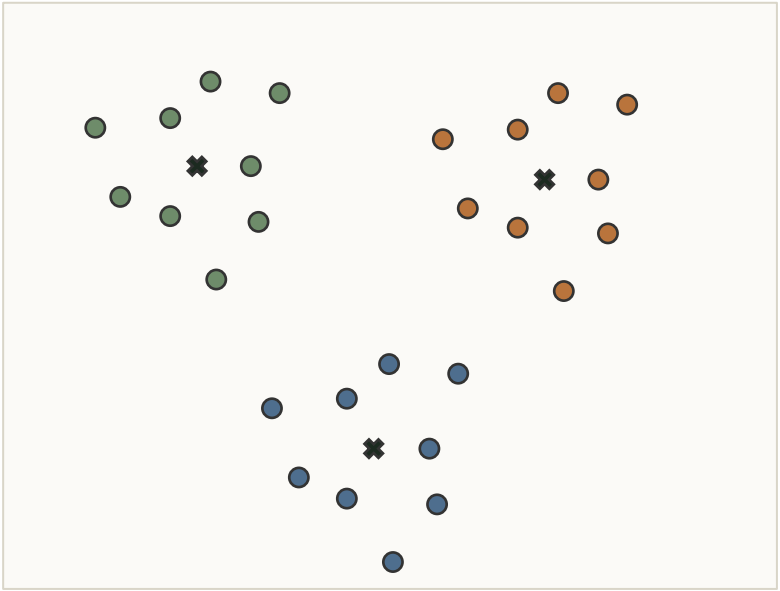
Unsupervised Learning

Structure without a target: k-means and the quality of a partition

Patterns without a prediction task

- Supervised learning finds patterns for a prediction task; unsupervised learning finds patterns with no target in mind.
- Take the iris data and hide the species column. Run a clustering model, and it recovers groups that were already there.
- k-means resembles k-NN in spirit — but there are no labels. The model produces the labels itself.
- Which is also the difficulty: there is no ground truth against which to score it.

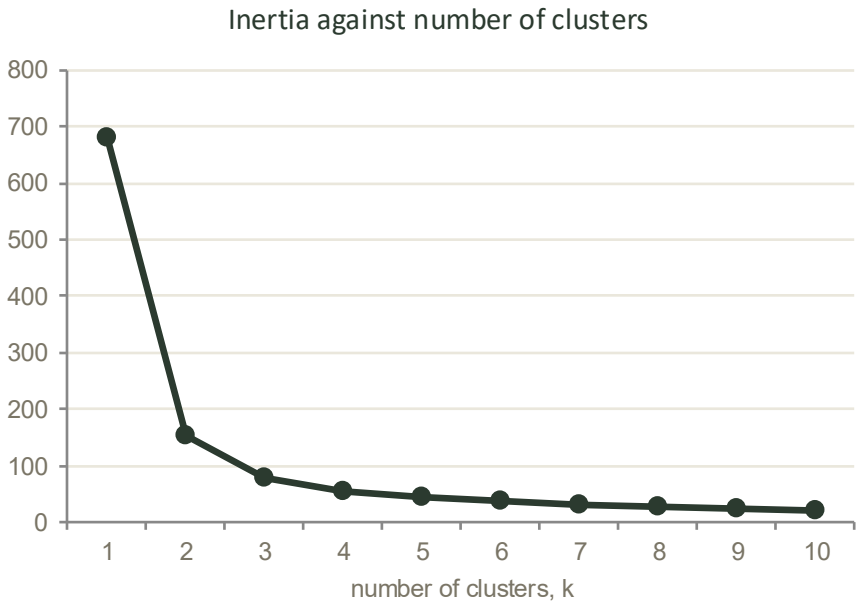
```
km = KMeans(n_clusters=3, n_init=10, random_state=0)
labels = km.fit_predict(X_scaled)
km.inertia_
```



× = CENTROID

Inertia and the elbow

- Inertia measures the quality of a partition: the total squared distance from each point to its centroid.
- Good clustering means tight clusters — low inertia. But inertia always falls as k rises, and k = n gives zero.
- So look for the elbow: the point past which an extra cluster buys almost nothing.
- It is a judgement, not a test. Say so when you report it.



Lab: a full pipeline — scale, cluster, profile the clusters substantively.

BEFORE THE NEXT SESSION

What to do with this

Lab	Re-run the notebook end to end on your own data. Break it deliberately once.
Read	Molina & Garip (2019); Hofman et al. (2021); Hindman (2015); Lones (2021), on the pitfalls.
Assignment 4	The shared task: preprocessing, model selection, training and testing, tuning, reported outputs.
Next	Advanced models and neural networks — Naive Bayes, SVM, trees and forests, PCA, MLP.

Credits: VanderPlas, Python Data Science Handbook · DataCamp, Supervised & Unsupervised Learning with scikit-learn